

Databricks Study

Self-learn project

Production-oriented workflow

1. **Starting an ETL pipeline in a development workspace (Databricks Free Edition is used as both the development and execution environment).**
2. **Data is ingested from a CSV stored in a Unity Catalog Volume.**

dir real-estate-project/

Real Production workflow

For this project, I used a Unity Catalog Volume as the landing zone for the raw CSV. In a production environment, the same ingestion logic could instead source data from cloud object storage (e.g., Amazon S3 or Azure Data Lake Storage) or enterprise databases such as SQL Server or Azure SQL Database.

I am currently using a single Databricks workspace but in the real-world there could be separate workspaces for development, test and deployment.



1. **Code is developed locally in VS Code**
2. **Bronze, Silver and Gold Delta tables are created using PySpark and Spark SQL.**
3. **Python visualisations are generated directly from the Gold layer using matplotlib/seaborn.**
4. **Version-controlled using GitHub. GitHub serves as the single source of truth for transformation scripts, version history, and project documentation.**

dir src/

bronze.py

- Reads raw CSV from a Unity Catalog Volume
- Creates the Bronze Delta table

silver.py

- Cleans and standardises the Bronze data
- Casts data types
- Produces the Silver Delta table

gold.py

- Creates business-ready aggregated Gold tables
- Uses Spark SQL for analytical transformations

visualisation.py

- Reads Gold tables
- Produces charts using matplotlib/seaborn

Multiple peer reviews would be done before bronze.py, silver.py, gold.py and visualisation.py are code merged in main branch.

Next, GitHub Action triggered upon receiving new commits to enable CI/CD that automates a few things (unit tests, code checking, deploy latest version to Databricks, trigger Databricks jobs, and even sending notifications to team members), instead of manually done on the left 'Production-oriented workflow'.

Step 1: Initialise workspace in Databricks and ensure it is linked to intended GitHub repo.

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

databricksFree Edition

Search data, notebooks, recents, and more...CTRL + P

workspace

Workspace

>

Home

>

Shared with me

>

Workspace

>

Favorites

>

Trash

Workspace > Users > yeechen92@gmail.com >

databricks_study

main

Open in editor

☆

Send feedback

Share

Create

Search

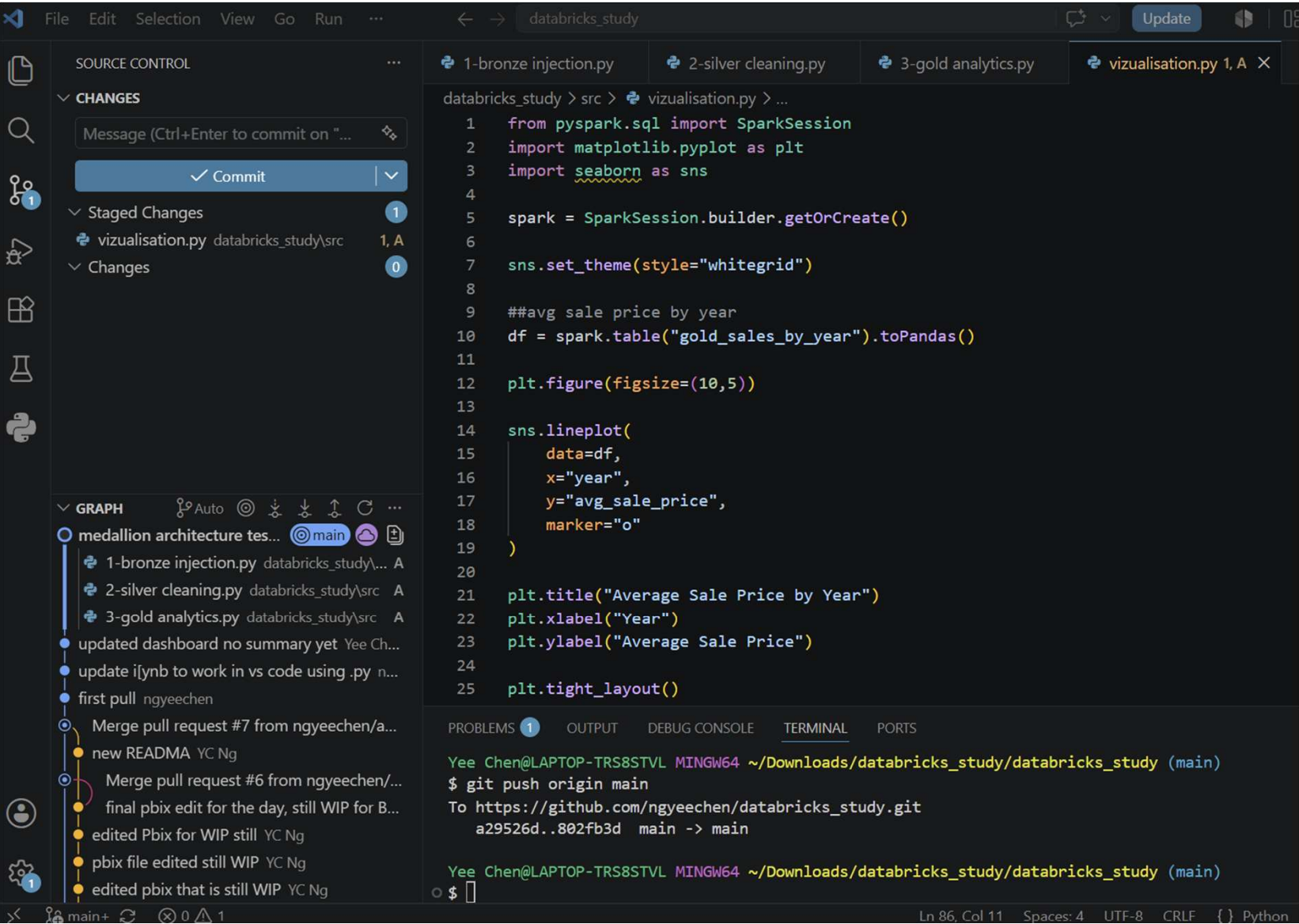
Type

Owner

Last modified

Name ↗	Type	Owner	Created at	Last updated at	
ref_git	Folder	yeechen92@gmail.com	Jul 08, 2025, 04:38 PM	Jul 08, 2025, 04:38 PM	
src	Folder	yeechen92@gmail.com	Jul 02, 2026, 02:50 PM	Jul 02, 2026, 02:50 PM	
databricks-Serverless Starter Warehouse.pbids	File	yeechen92@gmail.com	Aug 01, 2025, 03:46 PM	Aug 01, 2025, 03:46 PM	
★ prototypeForTestingOnly	Notebook	yeechen92@gmail.com	Aug 01, 2025, 04:08 PM	Jul 02, 2026, 04:08 PM	
Real Estate Property Transactions.csv	File	yeechen92@gmail.com	Aug 11, 2025, 10:44 AM	Aug 11, 2025, 10:44 AM	
Real Estate Property Transactions.xlsx	File	yeechen92@gmail.com	Aug 01, 2025, 03:46 PM	Aug 01, 2025, 04:06 PM	
WIP-YC demo-Real Estate data.pbix	File	yeechen92@gmail.com	Jul 02, 2026, 11:15 AM	Jul 02, 2026, 02:50 PM	

Step 2: After the creating the Git repo & initialising it in a local folder, code editing done locally and pushed to the repo.



Step 3: Pull from repo into the Databricks workspace and execute the scripts.



Free Edition

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

readAfteruploadIntoDBFS

1-bronze injection.py × +

Just now (27s)

Python

main

Last edit was now

Edit

Serverless

Schedule

Share

7

8

9

10

11

12

13

14

15

16

17

18

19

--

```
spark = SparkSession.builder.getOrCreate()

csv_path = "/Volumes/workspace/github/realstatedata/V3.csv"

bronze_df = (
    spark.read
        .option("header", "true")
        .option("inferSchema", "true")
        .csv(csv_path)
)

# Replace spaces in column names
bronze_df = bronze_df.toDF(
```

Add parameter

Output

Terminal

Debugger

pip logs

Bronze table created successfully.

> [See performance \(1\)](#)

 New

 Home

 Learn

 **Workspace**

 Recents

 Catalog

 Jobs & Pipelines

 Compute

 Discover

 Marketplace

SQL

 SQL Editor

 Queries

 Dashboards

 Genie Spaces

 Alerts

 Query History

 SQL Warehouses

 1-bronze injection.py  prototypeForTestingOnly  2-silver cleaning.py ×  3-gold analytics.py + 

 03:43 PM (49s) Python  main  Last edit was 18 minutes ago   Edit  Serverless  Schedule  Share 

```

26         .withColumn("Residential", trim(col("Residential")))
27         .withColumn("Face", trim(col("Face")))
28     )
29
30     silver_df.write \
31         .mode("overwrite") \
32         .saveAsTable("property_transactions_silver")
33
34     print("Silver table created successfully.")
35
36
37 if __name__ == "__main__":
38     run()

```

Output  Terminal  Debugger  pip logs  ^ ×

Silver table created successfully.

>  See performance (1) 

 New

 Home

 Learn

 Workspace

 Recents

 Catalog

 Jobs & Pipelines

 Compute

 Discover

 Marketplace

SQL

 SQL Editor

 Queries

 Dashboards

 Genie Spaces

 Alerts

 Query History

 SQL Warehouses



1-bronze injection.py



prototypeForTestingOnly



2-silver cleaning.py



3-gold analytics.py ×



03:45 PM (19s)

Python ▾



main



Last edit was 1 hour ago



Edit



Serverless ▾



Schedule



Share



49

```
num_rooms,
```

50

```
  COUNT(*) AS total_sales,
```

51

```
  AVG(Sale_Price) AS avg_sale_price
```

52

```
FROM silver
```

53

```
GROUP BY num_rooms
```

54

```
ORDER BY num_rooms
```

55

```
""")
```

56

57

```
print("Gold tables created successfully.")
```

58

59

60

```
if __name__ == "__main__":
```

61

```
    run()
```

Add parameter

Output

Terminal

Debugger

pip logs



Gold tables created successfully.



 See performance (5)

Step 4: Charts generated. Verify output. Edit the scripts to export using plt.savefig if needed for external reporting.

